Lab C: Advanced Shell Development

Lab C builds on top of the code infrastructure of the "shell" from Lab 2. Naturally, you are expected to use the code you wrote for Lab 2 in this lab and extend it. **This do-at-home lab may be done in pairs.**

Overview

In this lab you will enrich the set of capabilities of your shell by adding:

- **Job control** — manage running/suspended processes
- **Pipes** — connect the output of one command to the input of another
- **History** — save and recall previous commands

Note: All parts below, except for Part 1, are features that should be added to your shell as **one program** (in however many C functions you wish) that supports all these features together.

Part 0: Preparation

Step 1: History Mechanism

Check out the "history" mechanism in the Linux shell. Try it out in a Linux shell to understand how it works.

Step 2: Code Organization

Reexamine your command interpreter (shell) code from Lab 2. If not done so before, reorganize it to be as modular and extensible as possible.

Part 1: An Exercise in Pipes

Note: This part is independent of the shell and serves as preparation for implementing pipe commands in the shell. You should not use the LineParser functions in this task, nor read any command lines. However, you need to declare an array of "strings" containing all of the arguments and ending with a NULL pointer to pass to `execvp()`, similar to the one returned by `parseCmdLines()`.

Overview

In this task, you will learn how to create a pipeline between two processes. A pipeline requires creating a pipe and properly redirecting the standard output and standard input of processes.

Task: Write a short program called `mypipeline` which creates a pipeline of 2 child processes. You will implement the shell command line:

```
ps -xl | grep 5
```

Questions to consider:

- What does `ps -xl` do?
- What does `grep 5` do?
- What should their combination produce?

Implementation Steps

Follow these steps closely to avoid synchronization problems:

1. **Create a pipe.**

II **Fork a first child process (child1).**

III
- Close the standard output
- Duplicate the write-end of the pipe using `dup()` (see `man dup`)
- Close the original file descriptor
- Execute `ps -xl`

IV **In the child1 process:**

V **In the parent process:** Close the write end of the pipe.

VI **Fork a second child process (child2).**

VII
- Close the standard input
- Duplicate the read-end of the pipe using `dup()`
- Close the original file descriptor
- Execute `grep 5`

VIII **In the child2 process:**

IX **In the parent process:** Close the read end of the pipe.

X **Wait for the child processes to terminate** in the same order they were created.

Debugging

Add debug messages printed to `stderr` to understand the flow:

In the parent process:

- Before forking: `(parent_process>forking…)`
- After forking: `(parent_process>created process with id: PID)`
- Before closing write end: `(parent_process>closing the write end of the pipe…)`
- Before closing read end: `(parent_process>closing the read end of the pipe…)`
- Before waiting: `(parent_process>waiting for child processes to terminate…)`
- Before exiting: `(parent_process>exiting…)`

In the 1st child process:

- `(child1>redirecting stdout to the write end of the pipe…)`
- `(child1>going to execute cmd: …)`

In the 2nd child process:

- `(child2>redirecting stdin to the read end of the pipe…)`
- `(child2>going to execute cmd: …)`

Testing Variations

Test how the following changes affect your program:

I Comment out **step 4** (do not close the write end of the pipe in parent). Compile and run. (See `man 7 pipe`)

II Undo. Comment out **step 7** (do not close the read end of the pipe in parent). Compile and run.

III Undo. Comment out both **step 4 and step 8**. Compile and run.

Document your observations.

Part 2: Implementing a Pipe in the Shell

Having learned how to create a pipe between 2 processes in Part 1, implement pipeline support in your shell. You will support pipelines with **one pipe and 2 child processes**.

Overview

Your shell must now run commands like:

```
ls | wc -l
```

This counts the number of files/directories in the current working directory.

Important: The write-end of the pipe needs to be closed in all processes, otherwise the read-end will not receive EOF.

Implementation Notes

I **cmdLine parsing:** The line parser automatically generates a list of `cmdLine` structures for pipelines. For example, parsing `ls | grep .c` creates two chained `cmdLine` structures (one for `ls`, one for `grep`).

II
- **Error condition:** Redirecting the output of the left-hand-side process is an error (nothing would go into the pipe)
- **Error condition:** Redirecting the input of the right-hand-side process is an error (the pipe output is wasted)
- **Valid:** `cat < in.txt | tail -n 2 > out.txt` (input redirection on left, output on right)
- When errors occur, print an error message to stderr without creating any processes.

III **I/O Redirection + Pipes:** Your shell must still support all previous features from Lab 2, including I/O redirection.

IV **Memory:** Keep your program free of memory leaks.

Part 3: Process Manager

Every program run in your shell executes as a process. You will add an internal " process manager" to manage all processes forked by your shell.

The process manager provides these operations:

- `procs` — prints current processes (running, suspended, and "freshly" terminated)
- `wakeup <pid>` — wakes up a sleeping process (SIGCONT)
- `stop <pid>` — suspends a running process (SIGSTOP)
- `ice <pid>` — terminates a running/sleeping process (SIGINT)

- `nuke <gid>` — terminates a process group (SIGKILL)

Part 3.A: Process List

Representation

Create a linked list to store information about running/suspended processes. Each node is a `struct process`:

```
typedef struct process{
    cmdLine* cmd;                         /* the parsed command line*/
    pid_t pid;                            /* the process id running the co
mmand*/
    int status;                           /* status of the process*/
    struct process *next;                 /* next process in chain */
} process;
```

The `status` field can have these values:

```
#define TERMINATED  -1
#define RUNNING 1
#define SUSPENDED 0
```

Implementation

Implement these functions:

- - Receives a process list, command, and process ID
  - Inserts at the beginning of the list (process_list is a pointer to pointer)
- **`void addProcess(process** process_list, cmdLine* cmd, pid_t pid)`**
- - Prints the processes in this format:
- **`void printProcessList(process** process_list)`**
- **Add support for `procs` command** — Your shell should accept the user typing `procs` and display the process list.

Example Session

```
#> sleep 3
#> procs
PID           STATUS     Command
14952         Terminated  sleep 3
#>

#> sleep 5&
#> procs
PID           STATUS     Command
14962         Running    sleep 5
#> # (wait for process to finish)
#>
#> procs
PID           STATUS     Command
14962         Terminated  sleep 5
#>
```

Part 3.B: Updating the Process List

Implement these additional functions:

- - Free all memory allocated for the process list
- `void freeProcessList(process* process_list)`
- - Check if each process has finished
  - Use `waitpid()` with `WNOHANG` flag (does not block)
  - Returns -1 if no process with given ID exists
  - **Important:** Read `man 2 waitpid` to learn how to detect SIGSTOP, SIGCONT, and SIGINT
- `void updateProcessList(process **process_list)`
- - Find the process with given ID in the list
  - Change its status to the received status
- `void updateProcessStatus(process* process_list, int pid, int status)`
- - Call `updateProcessList()` at the beginning
  - If a process was "freshly" terminated, delete it after printing (show updated status, then remove dead processes)
- **Update `printProcessList()`:**

Part 3.C: Manipulating the Processes

Add support for all these commands:

- `procs` — prints all processes (running, suspended, and terminated)
- `wakeup <pid>` — wakes up a sleeping process (SIGCONT)
- `stop <pid>` — suspends a running process (SIGSTOP)
- `ice <pid>` — terminates a running/sleeping process (SIGINT)
- `nuke <gid>` — terminates a process group (SIGKILL)

Implementation:

- Use `kill()` (see `man 2 kill`) to send signals
- Check if `kill()` succeeded and print appropriate messages
- Update the process status in the process_list

Test Scenario

Use your looper code from Lab 2 to test:

```
#> ./looper&
#> ./looper&
#> ./looper&
#> procs
PID       STATUS       Command
18170     Running      ./looper
18171     Running      ./looper
18174     Running      ./looper

#> ice 18170
#> Looper handling SIGINT

#> stop 18174
#> Looper handling SIGSTOP
#> procs
PID       STATUS       Command
18170     Terminated   ./looper
18171     Running      ./looper
18174     Suspended    ./looper

#> wakeup 18174
#> Looper handling SIGCONT
```

```
#> wakeup 18171
#> Looper handling SIGCONT
#> procs
PID       STATUS       Command
18171     Running       ./looper
18174     Running       ./looper
```

Part 4: Adding the History Mechanism

Add a history mechanism to your shell to save and recall previous commands.

Overview

Your shell should keep **HISTLEN** previous command lines in a **circular queue**, where `HISTLEN = 10` (default).

Key points:

- Each entry contains a command-line number (starting at 1) and the unparsed command line
- When full, the oldest entry is removed and the new one is added at the end
- Implement in O(1) time by keeping indices of the beginning and end

User Commands

The user can now perform these shell commands (not as processes):

- `history` — print all history entries with their numbers
  - Must parse it again before executing
- `!!` — retrieve the last non-history command line, add it to history, and execute it
  - n is the command number (not the array index)

    - If n is invalid or doesn't exist, print an error message and ignore
- `!n` — retrieve command number n, add it to history, and execute it

Important Notes

- Keep the **UNPARSED** command lines in history (not the parsed version)
- History should work **on top of all other features** (pipes, redirection, etc.)
- This should be straightforward if your code is well-designed

---

Submission

Submit a zip file named:

- `[student-id-num].zip` (single submission), or
- `[student1-id_student2-id].zip` (pair submission)

The zip file must contain:

```
mypipeline.c        – Implementation of Part 1
myshell.c           – Shell with all features (Parts 2, 3, 4)
makefile            – Builds "mypipeline" and "myshell" executables
```